

Activity Briefing:

- Let's say you've figured out how to solve a problem, now it's time to code it up! This is implementation, and it is *hard*.
- Implementation used to be something we didn't used to explicitly teach, but we should, since your ability to do well in exams is min(implementation, problem solving).
- Like problem solving, it's something picked up mostly by practice, but there are some key ways of thinking that can act as a force multiplier.
- This session seeks to give you some simple methods to make solutions that are easy to implement and easy to debug (simple!).

Common problem 1: Starting to code too early

- *Misconception: If I'm not coding up my solution, I'm wasting time.*
- **Pen & Paper:** Most extreme form comes from the ACM style of competitions (3 people, 1 keyboard). It's a bit overkill for you, but there are still some useful takeaways.
 - You type faster than you think! And sometimes, you need to think!
 - You can get lost in details and lose track of the overall algorithm.
 - Useful to make sure you haven't glossed over a part of the algorithm you don't know how to do.
 - 5 minutes of planning saves 10 minutes of implementation. Or more!

Planning out a solution:

- **Address Complexity:** Break up your solution into simple parts (simple functions, isolated sections, more loops).
 - Pseudocode test: Write down the steps of your algorithm in enough detail that you could do each step with minimal thinking (either on paper, or on the computer).
 - This is a nice ideal to shoot for, but you'll make mistakes (forget stuff, more complex than you thought etc.)
 - *Don't be afraid to hop off the keyboard when you need to!*
 - Easier to code up lots of parts that do simple things than few parts that do complex things.
 - Simple things are much easier to debug!!!!
 - Still feeling too complex? Might be worth seeking a more elegant solution. Even if you don't get anything, just having thought more about the problem can help out implementation.
- **Premortem:** Think about what you're likely to get wrong when you try to code this up.
 - Be wary of:
 - Things that are new, that you've never done before
 - Things similar, but different to something you've done before
 - Things you've messed up in the past
 - How many edge cases are there to think about?
 - Do you have to worry about doing something different for $N = 1$? $N = 0$?

- Are they handled elegantly? Sometimes finding a way to gracefully handle $N=1$ and especially $N=0$ can improve understanding of how something generalises, and yield much nicer implementations.
 - Gut feeling! Are there parts that feel complicated for you?
 - NB: Even though I present this as K sequential steps, in practice you'll jump around these steps as you go.
- **Implementation**
 - If you can't describe what a function does off the top of your head, break it up!
 - If it's not obvious immediately (which it won't be if you're new to informatics), comment it.
 - Avoid mutation. Prefer to make a new variable/array than to modify something that already exists.
 - Easier to remember what a variable does if it doesn't change!
 - Make sure you aren't blowing out your time complexity in doing this (e.g. copying a whole array to change one element)
 - **Invariants**
 - *What does your function promise to do, and what does it need to fulfill that promise?*
 - *What do you promise is complete once you're up to this section of code?*
 - Handle special cases early. Then you never have to think about them again.
 - Can you check your assumptions quickly? Assert it!
 - **DRY:** If you find yourself copy pasting code around, you should probably seek to simplify (turn it into a function!)
 - Copypasting and changing numbers is prone to error, checking for bugs is a nightmare, fixing a bug in one part means you need to remember to fix it in all the parts etc.
- **Debugging :(**
 - Printf debugging: Print out the state of certain variables over time, and you can track down the exact moment something is not what you expected.
 - Why did it stop being what you expected? What variables have the wrong values in them at this moment?
 - Rubber duck testing: Go through parts of your code and try to convince an imaginary, skeptical person that your code does what it says it should.
 - The more structured your code (in line with the principles above), the easier this part will be!
 - Test unusual cases
 -

By no means, is this an exhaustive list of techniques (there are a lot of them!). Some techniques (test driven development) Not specific to informatics by the way.

Lab exercise:

Find a problem you solved and write down what you struggled with, what you could do better, what bugs there were.

Closing out: The Postmortem

- Insert something about postmortem culture at Google here?
- See failure as a learning opportunity, don't blame yourself!
- Notice things that you code up over and over again. It's worth distilling those in your mind and standardising the way you do them each time. Faster and prevents bugs.
- What similar problems have you seen before?
- If something is annoying or frustrating, *there's probably a better way to do it.*
- Look at other people's solutions and see if you can learn anything. Is their code cleaner than yours?

Compiler Flags: <https://codeforces.com/blog/entry/15547>